SIMATS ENGINEERING

COURSE NAME: INTERNET PROGRAMMING **P.GOVARDHAN REDDY**
COURSE CODE: CSA4303 **marks : 78**

Assessment Tool 2 - Data Interpretation (Medium)

Student Name	P Govardhan Reddy
Register No.	192311249
Course Name	Internet Programming
Course Code	CSA4303
Course Outcome	CO-1 (L2): Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery.
GitHub Link	https://github.com/vinaykumar-bille/Internet_programming.git

RUBRICS FOR CALCULATION:

Criteria	**eigh ge**	**vel 4 (Exemplary)**	**l 3 (Proficient)**	**Level 2 Developing)**	**Level 1 eginning)**	
derstanding of Data	5%	urately interprets all a patterns, trends, and relationships	terprets data ectly with minor gaps	Partial derstanding; ses key trends	interprets or fails to erstand data	17
Analysis	5%	vides deep analysis ith strong logical reasoning and justification	Provides onable analysis with some ustification	asic analysis with weak reasoning	meaningful analysis provided	17
pplication of Concepts	0%	Effectively applies relevant concepts/tools to interpret data accurately	plies concepts h minor errors	Limited application; uires guidance	ble to apply epts to data	18
Conclusion	0%	Draws clear, meaningful conclusions with strong hts and implications	rovides valid nclusions with derate insights	nclusions are gue or weak	clear or valid onclusions	17

Clarity	0%	sents interpretation early using proper cture, visuals, and terminology	erally clear with minor issues	acks clarity or per structure	Poor sentation and difficult to nderstand	9

Question 1: HTTP Response Analysis

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150 Connection: keep-alive

1. Incorrect or invalid fields

Content-Length: -150	Content-Length must be a non-negative decimal integer (number of body bytes). A negative value is syntactically invalid (RFC 9110 / RFC 9112).	**Critical**
Date: Mon, 01 Apr 2026	1 April 2026 falls on a **Wednesday**, not Monday. The day-name and the calendar date contradict each other. The rest of the IMF fixdate format and the GMT zone are valid.	Moderate
Content-Type: text/html	Valid, but no charset parameter, so the browser has to guess the encoding.	Minor
Request line / Connection	HTTP/1.1 200 OK and Connection: keep-alive are syntactically valid.	OK

2. Issue with Content-Length and its impact

Content-Length tells the client exactly how many bytes of message body to read. It is the framing mechanism that marks where one response ends and the next begins on a persistent connection. A negative value is meaningless, so the message framing is broken.

- **Parsing failure:** a compliant client must treat the response as invalid and may discard it or show an error instead of the page.
- **Keep-alive risk:** with Connection: keep-alive the connection is reused, so a wrong length can make the client mis-read where the next response starts (response desynchronisation). • **Security:** inconsistent length handling between proxies, caches and servers is the basis of HTTP request/response smuggling and cache-poisoning attacks.
- **User impact:** truncated or blank pages, hanging loads, wasted connections and unreliable caching.

3. Does the status code match the response?

No - not consistently. A 200 OK status promises a successful, well-formed response, but the invalid Content-Length shows the response itself is malformed. If the body was generated correctly, 200 is appropriate once the headers are fixed. If the server failed while producing the content, it should return 500 Internal Server Error (or 502/504 if a gateway or upstream was at fault) instead of claiming success.

4. Corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
```

Connection: keep-alive

• Set Content-Length to the real byte size of the body (150 is an example), or drop it and use Transfer-Encoding: chunked for dynamic content.

• Fix the Date day-name to **Wed** (or generate it automatically from the system clock). • Declare the charset, and validate headers in server middleware / testing tools such as curl -I or browser DevTools.

Question 2: DOM Structure Analysis

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

1. Missing or improperly closed elements

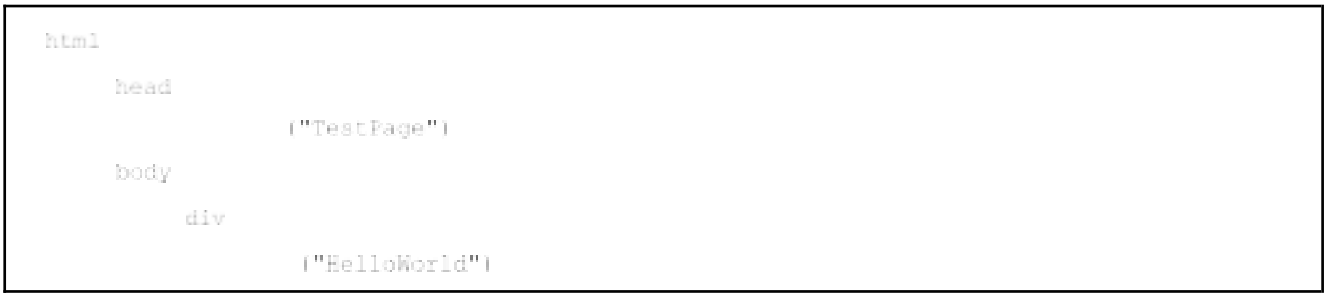
• **The** **element has no closing tag** (is missing) before .

• No <!DOCTYPE html> declaration, so browsers may fall back to quirks mode.

• No lang attribute on <html> and no <meta charset> in <head> (best-practice gaps). • Content text *HelloWorld* and title *TestPage* have no space (likely intended "Hello World" / "Test Page").

2. How the browser interprets the structure

As HTML, the end tag of is optional, so the parser applies **error recovery**: when it meets it implicitly closes the open paragraph first. The resulting DOM is:



If the file is served as **XHTML/XML** (application/xhtml+xml), there is no recovery: the document is not well-formed and the browser shows an XML parsing error instead of the page.

3. Impact on rendering and layout

• In HTML mode the visible result is normally the same: *HelloWorld* appears as a block-level paragraph inside the div with default margins.

- Missing DOCTYPE can trigger quirks mode, which changes box-model and spacing behaviour

across browsers.

- Relying on implicit closing becomes fragile when more content follows (stray nesting, unexpected styling, CSS selectors and scripts behaving differently).
- In XHTML/XML mode the page fails to render completely.

4. Corrected, valid version

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
    </div>
  </body>
</html>
```

Question 3: GET vs POST in Login System

GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

1. Differences in data transmission

Where data travels	Appended to the URL as a query string	Sent in the HTTP request body
Visibility	Shown in address bar, browser history, bookmarks, server and proxy logs, Referer header	Not shown in the URL or history; still plain text unless HTTPS is used
Caching	Can be cached and bookmarked	Not cached by default
Length limit	Limited by URL length (about 2-8 KB, browser/server dependent)	Practically no limit
Intended purpose	Safe, idempotent data retrieval	Submitting data / changing server state

2. Which method is more suitable for login, and why

POST is the suitable method. Credentials are kept out of the URL, so they do not leak through browser history, bookmarks, shared links, server access logs, proxy logs or the Referer header. POST also matches the semantics of login, which is an action that creates a session rather than a simple data lookup.

•

3. Security risks of using GET for authentication

• **Credential exposure:** the password (pass=1234) is stored in plain sight in history, logs and analytics tools.

• **Shoulder surfing and sharing:** anyone seeing the screen or a copied link sees the username and password.

• **Referer leakage:** the full URL can be sent to third-party sites when the user follows an external link. **Caching and replay:** proxies and browsers may cache or replay the request, enabling repeated or forged logins.

• **CSRF-friendly:** a login or other state-changing action triggered by a simple link or image tag is easier to abuse.

4. Best practices for secure login

• Always use **HTTPS/TLS**; POST alone is not encryption.

• Send credentials by POST only, and never put secrets in URLs or log them.

• Hash passwords with a slow salted algorithm (bcrypt, scrypt, Argon2); never store them in plain text. • Add CSRF tokens, secure session cookies (HttpOnly; Secure; SameSite) and regenerate the session ID after login.

• Apply rate limiting, account lockout / CAPTCHA and multi-factor authentication; use generic error messages ("Invalid username or password").

• Validate and sanitise all input server-side to prevent injection attacks; use autocomplete and type="password" properly in the form.

```
<form action="/login" method="POST">
    <input type="text" name="user" required>
    <input type="password" name="pass" required>
    <input type="hidden" name="csrf_token" value="...">
    <button type="submit">Login</button>
</form>
```

Question 4: HTML Code Inspection

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Missing closing tags

<h1>	**Missing </h1>**	Required. Heading end tag is not optional.

•

	**Missing **	Technically optional in HTML5, but should always be written.
<img>	No closing tag needed	Void element in HTML5 (needs /> only in XHTML).
html, head, title, body	Properly closed	OK.

2. Issues with the <img> tag

- **No** alt **attribute**: required for accessibility (screen readers), SEO and as fallback text if the image fails; omitting it is an HTML validation error.
- **No** width/height: causes layout shift while the image loads.
- **Relative path** image.jpg: if the file is missing or in a different folder the browser shows a broken-image icon (HTTP 404).

  In XHTML the tag must be self-closed: .
- Because the unclosed is the container, the image ends up inline inside that paragraph (and, see below, inside the heading).

3. How browsers handle these errors

HTML parsers are forgiving and recover from errors instead of stopping. Here, the unclosed <h1> is the most serious problem: a start tag does *not* close an open heading, so the paragraph and image are parsed **inside** the <h1>. Everything stays open until </body> forces implicit closure. Resulting DOM:

```
body
 +-- h1
        +-- "Welcome" +--
        p
              +-- "This is a paragraph"
              +-- img[src=image.jpg]
```

- **Visible effect:** the paragraph and image inherit the large, bold heading style, and the page outline/accessibility tree is wrong (the whole content is read as one heading).
- In strict **XHTML/XML** mode, the same code is a fatal well-formedness error and the page is not displayed.
- The page may still pass visual checks, but fails W3C validation, so the problems stay hidden until the layout grows.

4. Corrected version

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Sample</title>
</head>
<body>
   <h1>Welcome</h1>
   <p>This is a paragraph</p>
   <img src="image.jpg" alt="Sample illustration" width="400" height="300">
</body>
</html>
```

Question 5: Browser-Server Communication Flow

-

1. Interpretation of each step

1	User enters a URL	The browser parses the URL into scheme (https), host (www.example.com), port, path and query. It checks HSTS, history and cache before going to the network.
2	DNS lookup occurs	The hostname is resolved to an IP address via browser/OS cache, then the recursive resolver, root, TLD and authoritative name servers.
3	HTTP request is sent	A TCP connection (or QUIC for HTTP/3) is opened, followed by the TLS handshake for HTTPS. The browser then sends a request such as GET /index.html HTTP/1.1 with Host, cookies and Accept headers.
4	Server processes request	The web server (Nginx/Apache) routes the request, runs application logic, queries databases or caches and builds the content.
5	Response is returned	The server sends a status line (e.g. 200 OK), headers (Content-Type, Content Length, Cache-Control) and the body (HTML).

6	Browser renders page	The browser parses HTML into the DOM, CSS into the CSSOM, fetches sub-resources (CSS, JS, images), builds the render tree, then performs layout, paint and compositing.

Flow summary:

URL DNS TCP/TLS HTTP Request Server Response Render

2. Where delays can occur

DNS lookup	Uncached lookups, slow or distant resolvers, multiple lookups for many domains.
Connection setup	TCP three-way handshake and TLS negotiation add round trips, worst for far-away servers.
Network transfer	Latency, limited bandwidth, packet loss, large uncompressed payloads.
Server processing	Slow database queries, heavy backend code, overloaded or cold-start servers.
Rendering	Large render-blocking CSS/JS, many images and third-party scripts, complex layouts.

3. Role of DNS

DNS (Domain Name System) works as the Internet's phone book: it translates human-readable domain names into the IP addresses that routers use to reach servers. Without it the browser could not locate the server. DNS also enables load balancing and geo-routing (returning the nearest server or CDN node), failover and easy server migration without changing the URL. Because it is the very first network step, a slow or failing DNS lookup delays or blocks the entire page load ("server not found" errors). **4.**

Optimisations to improve performance

DNS	Use fast DNS providers, higher TTLs, dns-prefetch / preconnect hints, and reduce the number of distinct domains.
Connection	HTTP/2 or HTTP/3, TLS 1.3, keep-alive and connection reuse, OCSP stapling.
Delivery	CDN to serve content close to users; Gzip/Brotli compression; browser and server caching (Cache-Control, ETag).
Server	Optimise database queries and indexes, use caching (Redis), server-side caching and horizontal scaling.
Front-end	Minify and bundle CSS/JS, defer or async scripts, lazy-load and compress images (WebP/AVIF), inline critical CSS.

Conclusion

Well-formed headers, valid and properly nested markup, secure request methods and an optimised request flow are the foundations of reliable web delivery. Detecting such errors early by validation and inspection tools improves correctness, security, accessibility and performance.